Analisi della Semantica

1 Gestione dello scope

La gestione dello scope in Pike è affidata alla classe `LLVMCodeGenerator`, che mantiene una tabella dei simboli per tracciare le associazioni tra nomi di variabili e i loro puntatori in memoria (istruzioni `alloca` di LLVM).

Lo scope è definito **a livello di funzione**. Quando inizia la generazione del codice per una funzione, la tabella dei simboli viene svuotata, garantendo che le variabili locali e i parametri siano **visibili solo all'interno della funzione corrente**. Non è implementato uno scope globale persistente per le variabili tra funzioni diverse.

1.1 Raggruppamento in funzioni

Un programma in Pike è costituito da una serie di elementi **top-level**. Questi possono essere:

- **Definizioni di funzione:** Blocchi di codice riutilizzabili con nome e parametri.
- **Statement isolati:** Istruzioni scritte al di fuori di qualsiasi funzione.

Durante la fase di trasformazione dell'AST, se vengono rilevati statement al di fuori delle funzioni, questi vengono raggruppati automaticamente in una funzione implicita denominata `main`, che funge da punto di ingresso del programma.

Esempio

```
defunc fun1(...) {...} # Funzione1
x := 1;     # Toplevel (main)
y := 2;     # Toplevel (main)
z := x + y; # Toplevel (main)
defunc fun2(...) {...} # Funzione2
```

1.2 Scope dell'iteratore del for

Un caso particolare di gestione dello scope avviene nei cicli `for`. Il compilatore gestisce lo **shadowing** delle variabili: se l'iteratore del ciclo ha lo stesso nome di una variabile

esistente, quest'ultima viene temporaneamente salvata e nascosta, per poi essere ripristinata al termine del ciclo.

In fase di generazione codice, la variabile iteratore (es. `i`) viene gestita nello stack e, al termine del ciclo, viene rimossa o ripristinata al valore precedente se oscurava una variabile esterna.

2 Inferenza di tipo

2.1 Variabili

Pike utilizza un sistema di tipizzazione statica implicita per le variabili locali. Quando viene incontrata un'assegnazione per una variabile non presente nella *symbol table*, il compilatore inferisce il tipo basandosi sull'espressione assegnata:

- Se l'espressione è un `NumberValueAst`, il tipo inferito è `ir.DoubleType`.
- Se l'espressione è un `StringValueAst`, il tipo inferito è `i8*` (puntatore a char). La variabile viene quindi creata nello stack con il tipo corrispondente.

2.2 Propagazione

Il tipo dei dati viene propagato attraverso l'AST durante la visita ricorsiva. Le operazioni successive utilizzano questi tipi per determinare quali istruzioni LLVM generare.

2.3 Funzioni

Per le funzioni, i tipi dei parametri e del valore di ritorno sono definiti esplicitamente.

- I parametri possono essere `num` o `str`.
- Il valore di ritorno può essere `num` o `void` (se assente o esplicitamente nullo).

3 Type checking

3.1 Operazioni aritmetiche e di confronto

Il compilatore assume che le operazioni matematiche (`+`, `-`, `*`, `/`) avvengano tra numeri.

Se un operando non è di tipo double (ad esempio una stringa), LLVM solleverebbe un errore durante la fase di costruzione dell'IR o di verifica del modulo, poiché non ci sono cast automatici espliciti prima di queste chiamate.

Quindi, non sono possibili operazioni tra stringhe.

3.2 Assegnazioni e ridefinizioni

Nelle assegnazioni:

- Se la variabile è nuova, assume il tipo del valore assegnato.
- Se la variabile esiste già, viene effettuata una `store` all'indirizzo esistente. LLVM richiede che il valore memorizzato corrisponda al tipo del puntatore allocato; in caso di mismatch (es. assegnare una stringa a una variabile numerica esistente), si verificherebbe un errore a livello di llvmlite.

3.3 Chiamate a funzione

Durante una chiamata a funzione, viene effettuato un controllo semantico sul numero degli argomenti. Inoltre, viene verificato che il nome chiamato corrisponda effettivamente a una funzione definita nel modulo globale.

3.4 Controllo di flusso

Le strutture di controllo (`if`, `for`) richiedono che la condizione sia valutabile come un valore booleano. Poiché i booleani sono rappresentati come `double`, il controllo semantico consiste nel confrontare il risultato della condizione con 0.0. Questo implica che l'espressione di condizione deve restituire un tipo compatibile con il confronto floating-point.

3.5 Valore di ritorno

Il compilatore verifica la coerenza tra il tipo dichiarato della funzione e l'istruzione di ritorno. Se la funzione ritorna un numero, il valore dell'espressione deve essere compatibile con `double`.